**EXERCISE 15.2**

Implement `count`. It should emit the number of elements seen so far. For instance, `count(Stream("a", "b", "c"))` should yield `Stream(1, 2, 3)` (or `Stream(0, 1, 2, 3)`, your choice).

```
def count[I]: Process[I,Int]
```

**EXERCISE 15.3**

Implement `mean`. It should emit a running average of the values seen so far.

```
def mean: Process[Double,Double]
```

Just as we've seen many times throughout this book, when we notice common patterns when defining a series of functions, we can factor these patterns out into generic combinators. The functions `sum`, `count` and `mean` all share a common pattern. Each has a single piece of state, has a state transition function that updates this state in response to input, and produces a single output. We can generalize this to a combinator, `loop`:

```
def loop[S,I,O](z: S)(f: (I,S) => (O,S)): Process[I,O] =
  Await((i: I) => f(i,z) match {
    case (o,s2) => emit(o, loop(s2)(f))
  })
```

**EXERCISE 15.4**

Write `sum` and `count` in terms of `loop`.

### 15.2.2 Composing and appending processes

We can build up more complex stream transformations by *composing* `Process` values. Given two `Process` values `f` and `g`, we can feed the output of `f` to the input of `g`. We'll name this operation `|>` (pronounced *pipe* or *compose*) and implement it as a function on `Process`.[4] It has the nice property that `f |> g` *fuses* the transformations done by `f` and `g`. As soon as values are emitted by `f`, they're transformed by `g`.

[4] This operation might remind you of function composition, which feeds the (single) output of a function as the (single) input to another function. Both `Process` and `Function1` are instances of a wider abstraction called a *category*. See the chapter notes for details.

**EXERCISE 15.5**

*Hard:* Implement `|>` as a method on `Process`. Let the types guide your implementation.

```
def |>[O2](p2: Process[O,O2]): Process[I,O2]
```

We can now write expressions like `filter(_ % 2 == 0) |> lift(_ + 1)` to filter and map in a single transformation. We'll sometimes call a sequence of transformations like this a *pipeline*.

Since we have `Process` composition, and we can lift any function into a `Process`, we can easily implement `map` to transform the output of a `Process` with a function:

```
def map[O2](f: O => O2): Process[I,O2] = this |> lift(f)
```

This means that the type constructor `Process[I,_]` is a functor. If we ignore the input side `I` for a moment, we can just think of a `Process[I,O]` as a sequence of `O` values. This implementation of `map` is then analogous to mapping over a `Stream` or a `List`.

In fact, most of the operations defined for ordinary sequences are defined for `Process` as well. We can, for example, *append* one process to another. Given two processes `x` and `y`, the expression `x ++ y` is a process that will run `x` to completion and then run `y` on whatever input remains after `x` has halted. For the implementation, we simply replace the `Halt` of `x` with `y` (much like how `++` on `List` replaces the `Nil` terminating the first list with the second list):

```
def ++(p: => Process[I,O]): Process[I,O] = this match {
  case Halt() => p
  case Emit(h, t) => Emit(h, t ++ p)
  case Await(recv) => Await(recv andThen (_ ++ p))
}
```

With the help of `++` on `Process`, we can define `flatMap`:

```
def flatMap[O2](f: O => Process[I,O2]): Process[I,O2] = this match {
  case Halt() => Halt()
  case Emit(h, t) => f(h) ++ t.flatMap(f)
  case Await(recv) => Await(recv andThen (_ flatMap f))
}
```

The obvious question is then whether `Process[I,_]` forms a monad. It turns out that it does! To write the `Monad` instance, we have to partially apply the `I` parameter of `Process`—a trick we've used before:

```
def monad[I]: Monad[({ type f[x] = Process[I,x]})#f] =
  new Monad[({ type f[x] = Process[I,x]})#f] {
    def unit[O](o: => O): Process[I,O] = Emit(o)
```

```
    def flatMap[O,O2](p: Process[I,O])(
                      f: O => Process[I,O2]): Process[I,O2] =
      p flatMap f
  }
```

The `unit` function just emits the argument and then halts, similar to `unit` for the `List` monad.

This `Monad` instance is the same idea as the `Monad` for `List`. What makes `Process` more interesting than just `List` is that it can accept *input*. And it can transform that input through mapping, filtering, folding, grouping, and so on. It turns out that `Process` can express almost any stream transformation, all the while remaining agnostic to how exactly it's obtaining its input or what should happen with its output.

Implement `zipWithIndex`. It emits a running count of values emitted along with each value; for example, `Process("a","b").zipWithIndex` yields `Process(("a",0), ("b",1))`.

*Hard:* Come up with a generic combinator that lets you express `mean` in terms of `sum` and `count`. Define this combinator and implement `mean` in terms of it.

Implement `exists`. There are multiple ways to implement it, given that `exists(_ % 2 == 0)(Stream(1,3,5,6,7))` could produce `Stream(true)` (halting, and only yielding the final result), `Stream(false,false,false,true)` (halting, and yielding all intermediate results), or `Stream(false,false,false,true,true)` (*not* halting, and yielding all the intermediate results). Note that because `|>` fuses, there's no penalty to implementing the "trimming" of this last form with a separate combinator.

```
def exists[I](f: I => Boolean): Process[I,Boolean]
```

---

We can now express the core stream transducer for our line-counting problem as `count |> exists(_ > 40000)`. Of course, it's easy to attach filters and other transformations to our pipeline.